

Fruit+JNPF 低代码平台 — 物联网项目开发能力扩展评估（第一版）

文档性质：技术架构选型评审解读（非实施方案）
评估范围：机械加工/钢铁行业 MES · 智慧工地 · 智能手环
当前阶段：架构选型评审 → 审批后纳入三期统一开发
评估方法：源码全量扫描 + 三路并行深度探索 + 竞品/行业标准对标
源码基线：d:\liu202505v2 (.NET 6 / SqlSugar / Furion 衍生框架)

目录

- 第一章：评估范围与方法论
- 第二章：现有架构能力基线（源码事实）
- 第三章：需求 1-5 MES 业务层评估
- 第四章：需求 6 物联网实时通信评估
- 第五章：需求 7 架构决策 — 升级扩展 vs 另建框架
- 第六章：三期核心架构增强清单
- 第七章：低代码引擎 IoT 扩展评估
- 第八章：实施路线图与工期估算

第一章：评估范围与方法论

1.1 评估背景

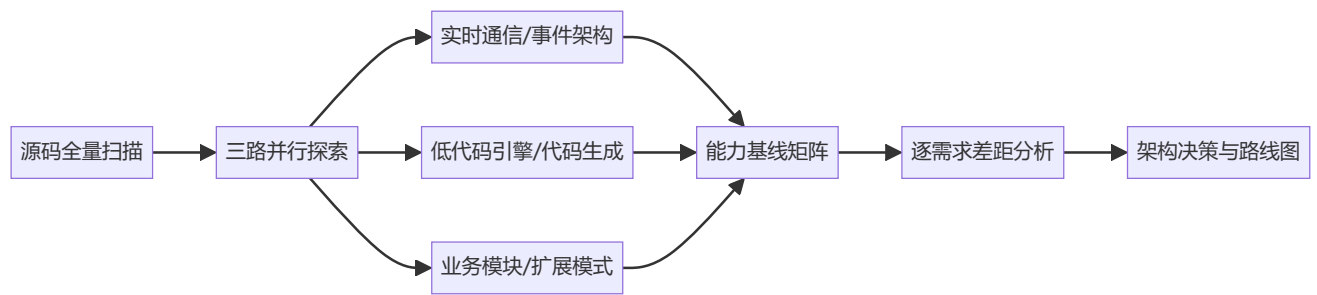
平台一期/二期聚焦 **通用办公 OA 低代码** 能力（表单/流程/消息/权限）。三期目标是在同一核心框架上支撑 **工业物联网（IIoT）** 类项目交付：

项目类型	典型场景
机械加工 MES	ERP 订单导入 → 工单排产 → BOM 配料 → 工序跟踪 → 用工单打印 → 工资核算
钢铁行业 MES	炼钢炉传感器 → 轧钢操作岗 → 拉索操作岗 → 质检岗 → 秒级数据采集 → 实时大屏
智慧工地	塔吊/升降机人脸识别考勤 → 设备监控 → 安全告警 → 指令控制（吊臂移动）
智能手环	健康检测数据实时同步 → APP 展示 → 异常告警 → 主动触发检测指令

1.2 评估的 7 个功能需求

# 需求	领域
1 ERP 订单导入 + 工单排产	MES 业务
2 BOM 包材料配置管理	MES 业务
3 工单生产进度跟踪	MES 业务
4 工单→生产步骤→用工单	MES 业务
5 用工单打印存档（核发工资）	MES 业务
6 物联网数据网关 MQTT 秒级同步 + 实时大屏 + 设备控制指令	IoT 基础设施
7 升级扩展 or 另建 IoT 版低代码框架？	架构决策

1.3 评估方法



1.4 关键术语

术语	含义
MES	Manufacturing Execution System，制造执行系统
BOM	Bill of Materials，物料清单
APS	Advanced Planning & Scheduling，高级排产
MQTT	Message Queuing Telemetry Transport，物联网消息协议
TSDB	Time Series Database，时序数据库
OPC UA	Open Platform Communications Unified Architecture，工业通信协议
CEP	Complex Event Processing，复杂事件处理

本节核心表清单

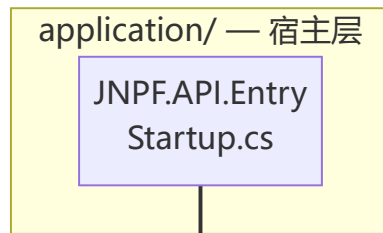
无（方法论章节）。

本节关键代码路径索引

无（方法论章节）。

第二章：现有架构能力基线（源码事实）

2.1 架构全景（四层结构）



modularity/ — 业务模块层 (15 模块)

system

workflow

visualdev

message

extend

visualdata

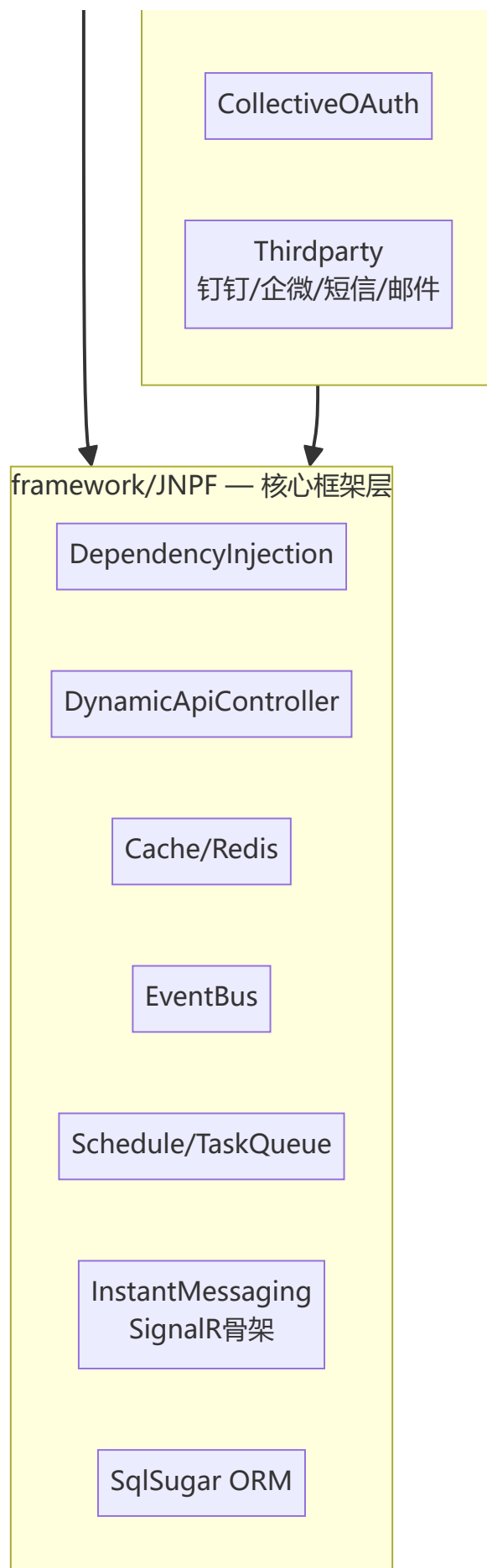
??? iot ???
(不存在)

infrastructure/ — 基础设施层

EventBus.RabbitMQ

WebSockets

```
graph TD; subgraph infrastructure_layer [infrastructure/ — 基础设施层]; direction TB; B["EventBus.RabbitMQ"]; C["WebSockets"]; end;
```



2.2 与物联网相关的能力矩阵

能力域	现有实现	成熟度	IoT 可用性	源码位置
WebSocket	IMHandler 原生 WS，按用户/租户分组推送	中	连接模型仅用 户维度，无设备维度	JNPF.Common.Core/Handlers/IMHandler.cs
SignalR	框架骨架 (MapHubAttribute + MapHubs())， 无任何 Hub 实现	骨架	需从零建 IoT Hub	framework/JNPF/InstantMessaging/Hub
EventBus	内存 + RabbitMQ (单队列 eventbus，无 Topic 路由)	中	可做内部事件解耦，不能替代 MQTT	Startup.cs L174-216
定时调度	AddSchedule + DbJobPersistence，但 ScheduleTaskService/PerformJob 入口被临时 return "" 禁用	框架完整/业务禁用	低频补采可用，秒级采集不适用	JNPF.TaskScheduler/TimeTaskService.cs
MQTT	零实现 。全库无 MQTTnet/Mqtt 引用	无	硬性缺失	—
时序数据库	零实现 。无 InfluxDB/TDengine/TimescaleDB	无	硬性缺失	—
设备管理	零实现 。无 Device/Sensor/Gateway 模型	无	硬性缺失	—
数据大屏	VisualData 7 个 Service，前端 ECharts， HTTP 轮询取数	中	无实时推送，需改造	JNPF.VisualData/ScreenService.cs
低代码引擎	VisualDev 表单+列表+流程，元数据驱动 CRUD	高	MES 表单可用，设备监控不适用	JNPF.VisualDev/RunService.cs
代码生成	Velocity 模板 8 类 (Vue2/3 + C#)，主从/子表/流程	高	可扩展 IoT 模板	wwwroot/Template/
模块扩展	三件套模式 (Service/Entitys/Interfaces)，DynamicApi 自动暴露	高	新增 mes/iot 模块零改框架	modularity/extend/
单据编号	BillRuleService 前缀+日期+流水号 +Redis 缓存	高	工单编号直接复用	BillRuleService.cs

能力域	现有实现	成熟度	IoT 可用性	源码位置
打印模板	PrintDevService HTML 模板 + SQL/数据接口取数	高	用工单打印直接复用	PrintDevService.cs
Excel 导入	NPOI + ExcelImportHelper + ExportImportDataHelper	高	ERP 订单导入直接复用	ExcelImportHelper.cs
流程引擎	8 个 Service + FlowTaskManager + FormType 1/2	高	工单审批可用	JNPF.WorkFlow/

2.3 关键发现摘要

#发现

- 1 MQTT、TSDB、设备管理**三项硬性缺失**
- 2 SignalR 有框架骨架但零 Hub 实现
- 3 RabbitMQ 仅单队列无 Topic 路由
- 4 ScheduleTask 入口被 return "" 禁用
- 5 大屏 blade_visual_record 表有 mqttUrl 列但后端未映射
- 6 低代码引擎是"表单+列表+流程" CRUD 模型
- 7 MES 所需的业务基础设施 (BillRule/PrintDev/Workflow/Import) **全部就绪**

影响

物联网实时通道不可用
需手动创建 IoT Hub
不能按设备/遥测/告警分 Topic
定时采集不可用 (需恢复)
Schema 超前、实现缺失
设备监控面板不适合套用 VisualDev
MES 可增量构建

本节核心表清单

表名	说明
BASE_BILL_RULE	单据编号规则
BASE_PRINT_TEMPLATE	打印模板
BASE_TIME_TASK	定时任务
BLADE_VISUAL_RECORD	大屏数据源 (含未映射的 mqttUrl)

本节关键代码路径索引

路径	说明
application/JNPF.API.Entry/Startup.cs	全部服务注册
framework/JNPF/InstantMessaging/	SignalR 骨架
infrastructure/JNPF.Extras.EventBus.RabbitMQ/	RabbitMQ 插件
modularity/common/JNPF.Common.Core/Handlers/IMHandler.cs	WebSocket 处理
modularity/visualdata/JNPF.VisualData/	大屏 7 个 Service
modularity/visualdev/JNPF.VisualDev/RunService.cs	低代码运行时引擎

第三章：需求 1-5 MES 业务层评估

3.0 总体判断

需求 1-5 可在现有架构上增量构建，无需重写任何核心组件。参照 modularity/extend/ 的 OrderService/ProductService 模式，在 modularity/ 下新增 mes/ 三件套模块即可。

3.1 需求 1：ERP 订单导入 + 工单排产

3.1.1 现有能力

能力	源码位置	可复用度
Excel 导入	ExcelImportHelper.ToDataTable → 行级校验 → 批量入库	直接复用
API 数据接口	DataInterfaceService (Type=3 API 操作) 可拉取 ERP REST 接口	直接复用
单据编号	BillRuleService.GetBillNumber("WorkOrderNo")	直接复用

3.1.2 需新建

组件	说明
WorkOrderEntity / MES_WORK_ORDER	工单主表 (订单号/产品/数量/计划开工/计划完工/状态/优先级)
WorkOrderservice	CRUD + 导入 + 排产状态机
排产算法	一期：简单优先级+甘特视图；复杂 APS 后期独立算法服务

3.1.3 排产能力分级建议

等级	算法	适用	复杂度
L1 手动排产	管理员拖拽甘特图调整顺序	小型车间	低 (前端为主)
L2 规则排产	按优先级+交期+产能自动排序	中型车间	中 (后端 Service)
L3 APS 排产	遗传算法/约束规划	大型工厂	高 (独立微服务)

建议三期先实现 L1+L2，L3 作为后续专项。

3.2 需求 2：BOM 包材料配置管理

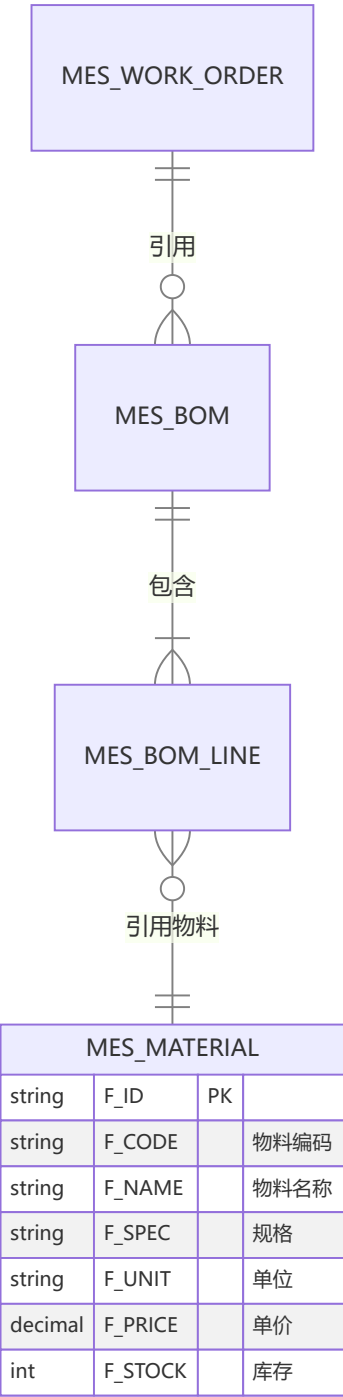
3.2.1 现有能力

ProductService + ProductEntryEntity 已实现主从物料结构，[Navigate(OneToMany)] 关联明细行。BOM 是其行业化扩展。

3.2.2 需新建

组件	说明
BomEntity / MES_BOM	BOM 主表 (产品编码/版本/状态)
BomLineEntity / MES_BOM_LINE	BOM 行 (物料编码/名称/用量/单位/损耗率/替代料)
BomService	CRUD + 版本管理 + 成本核算

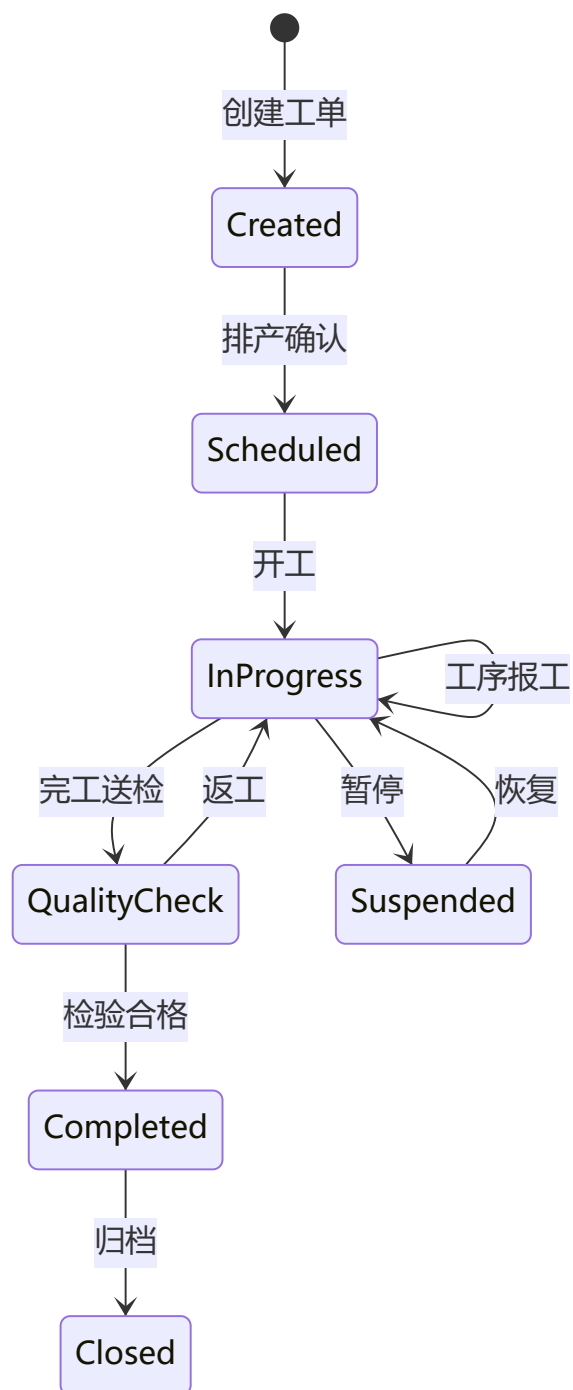
3.2.3 数据模型草案



3.3 需求 3：工单生产进度跟踪

3.3.1 设计思路

工单状态机 + 工序报工：



3.3.2 需新建

组件

MES_WORK_ORDER_PROCESS
MES_PROCESS_REPORT
ProductionProgressService

说明

工序路线表（工序号/工序名/标准工时/设备/岗位）
工序报工记录（实际开工/完工/数量/合格数/操作员）
进度汇总 + 与 Dashboard/大屏数据源对接

3.3.3 与流程引擎集成

工序审批（如质检确认）可绑定 FlowTaskManager（FormType=1 系统表单），审批通过后自动推进工单状态。

3.4 需求 4：工单 → 生产步骤 → 用工单

3.4.1 设计思路

用工单 (Labor Ticket) = 工单 + 工序 + 人员 + 工时：

表	说明
MES_LABOR_TICKET	用工单主表 (工单Id/工序Id/员工Id/计划工时/实际工时/计件数量/日期)

用工单由**工序报工**自动生成或手动录入，按人按日汇总即可计算工资。

3.4.2 与 BillRule 集成

```
// 用工单编号规则配置
// 前缀 LT-，日期 yyyyMMdd，流水号 4 位
entity.TicketNo = await _billRuleService.GetBillNumber("LaborTicketNo");
```

3.5 需求 5：用工单打印存档

3.5.1 现有能力

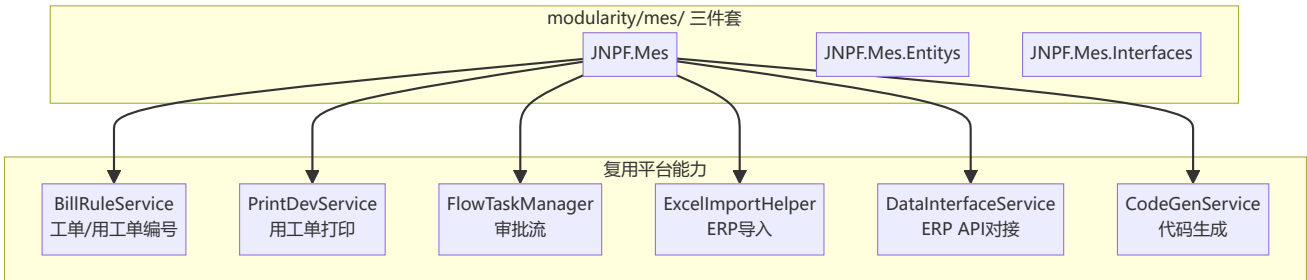
PrintDevService 已支持：

- 可视化 HTML 打印模板设计
- 数据源绑定 SQL (@formId 参数) 或数据接口
- 打印日志 (BASE_PRINT_LOG)
- 审批记录附带 (Join FlowTaskOperatorRecordEntity)

3.5.2 落地方式

1. 在系统管理「打印模板」中新建 **用工单打印模板**
2. SQL 数据源：SELECT * FROM MES_LABOR_TICKET lt JOIN MES_WORK_ORDER wo ON ... WHERE lt.F_ID = @formId
3. 前端调用 GET api/system/PrintDev/Data/{templateId}?formId={ticketId} 获取数据 + 模板
4. 浏览器端渲染打印，**无需改 PrintDevService 源码**

3.6 MES 模块整体架构



本节核心表清单

表名	说明
MES_WORK_ORDER	工单主表
MES_BOM / MES_BOM_LINE	BOM 主表/行
MES_MATERIAL	物料主数据
MES_WORK_ORDER_PROCESS	工序路线
MES_PROCESS_REPORT	工序报工
MES_LABOR_TICKET	用工单

本节关键代码路径索引

路径	说明
modularity/extend/JNPF.Extend/OrderService.cs	主从表 + 流程参考
modularity/extend/JNPF.Extend/ProductService.cs	BillRule + Navigate 参考
modularity/system/JNPF.Systems/System/BillRuleService.cs	单据编号
modularity/system/JNPF.Systems/System/PrintDevService.cs	打印模板
modularity/common/JNPF.Common/Security/ExcelImportHelper.cs	Excel 导入

第四章：需求 6 物联网实时通信评估

4.0 总体判断

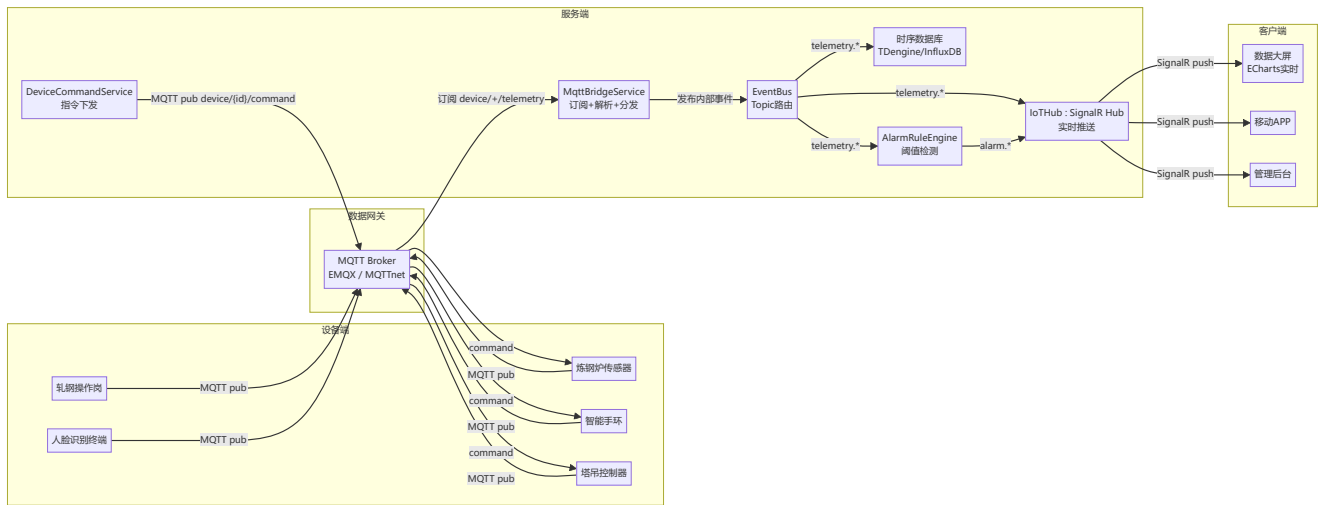
物联网实时数据通道是平台当前最大的能力缺口。需要在框架层和业务层同时新增 IoT 基础设施，但这些新增是旁路扩展，不需要重写现有架构。

4.1 需求拆解

需求 6 实际包含 5 个子能力：

子能力	说明	数据方向
C1 设备接入	MQTT 网关 + 设备注册/认证/心跳	设备 → 平台
C2 遥测采集	秒级传感器数据接收 + 存储	设备 → 平台 → TSDB
C3 实时大屏	数据实时跳动 + 异常高亮	平台 → 浏览器
C4 告警引擎	阈值检测 + 实时告警推送	平台 → 浏览器/APP
C5 指令控制	设备停机/手环检测/塔吊移动	平台 → 设备

4.2 端到端数据流设计



4.3 五个子能力的差距分析

C1: 设备接入 (MQTT 网关)

项	现状	目标	差距
协议	无 MQTT	MQTT 3.1.1/5.0	从零建设
Broker	无	EMQX (推荐) 或 MQTTnet 内嵌	从零建设
设备认证	无	一机一密 / 证书 / Token	从零建设
Topic 规范	无	device/{deviceId}/telemetry / command / status	从零设计
设备影子	无	期望状态 + 实际状态 JSON 文档	从零建设

技术选型建议:

方案	优点	缺点	推荐场景
EMQX (外部 Broker)	百万级连接、规则引擎、集群、Dashboard	额外运维	万级设备以上
MQTTnet (内嵌 Broker)	.NET 原生、单体部署简单	性能上限、无规则引擎	千级设备以下
阿里云/华为云 IoT Hub	全托管	云厂商绑定、成本	快速 PoC

推荐: 三期先用 EMQX (开源版), 后端通过 MQTTnet Client 订阅/发布。

C2: 遥测数据采集与存储

项	现状	目标	差距
接收	无	MqttBridgeService 订阅 MQTT + 反序列化	从零建设
存储	仅 SQL Server 关系库	时序数据库 (高频写入/时间范围查询/自动降采样)	从零建设
历史查询	无	按设备/时间/测点查询 + 聚合	从零建设

时序数据库选型:

方案	优点	缺点	推荐
TDengine	国产、SQL 兼容、超级表/子表模型、高压压缩	生态较 InfluxDB 小	推荐 (国产化 + SQL 友好)
InfluxDB	全球生态最大、Flux 查询语言	商业版收费、Flux 学习曲线	备选
TimescaleDB	PostgreSQL 扩展、SQL 原生	依赖 PG	已用 PG 时考虑

方案	优点	缺点	推荐
SQL Server + 冷热分离	无需新增组件	高频写入性能差、存储成本高	仅极低频（分钟级）场景

推荐：TDengine（SQL 兼容降低学习成本 + 超级表天然适配设备-测点模型）。

C3：实时大屏

项	现状	目标	差距
数据推送	HTTP 轮询	SignalR Hub 实时推送	需激活 SignalR + 建 IoT Hub
前端图表	ECharts 静态	ECharts 实时刷新 (setInterval → SignalR onMessage)	前端改造
告警高亮	无	异常值红色闪烁 + 声音提醒	前端新增
MQTT 直连	blade_visual_record.mqtturl 列存在但后端未映射	补全映射或改为 SignalR 中转	选型决策

推荐：服务端 SignalR 中转（而非浏览器直连 MQTT），原因：

1. 浏览器直连 Broker 暴露地址和凭证
2. 服务端可做数据聚合/降频/鉴权后再推送
3. 复用 JNPF 现有 JWT 认证体系

C4：告警引擎

项	现状	目标
规则配置	无	管理界面配置阈值（温度>1500°C → 高温报警）
检测方式	无	流式匹配（CEP 或简单 if-then）
通知	站内信 + WebSocket	站内信 + SignalR + 钉钉/企微 + APP 推送 + 声光告警

建议：三期用 简单规则引擎（配置化 if-then + EventBus），不上 Flink/Drools 等重型 CEP。

C5：指令控制

项	现状	目标
下发通道	无	MQTT publish device/{id}/command
指令格式	无	JSON { "action": "stop", "params": {...}, "requestId": "..." }
回执	无	设备 publish device/{id}/command/response, 平台匹配 requestId
权限	无	操作员需具备设备控制权限 + 流程审批（危险操作）
超时	无	指令超时未回执 → 告警

典型控制场景：

场景	指令	QoS
设备停机	{ "action": "shutdown" }	QoS 1（至少一次）
手环检测	{ "action": "measure_health" }	QoS 1
塔吊移动	{ "action": "move", "params": { "axis": "left", "distance": 0.5 } }	QoS 2（精确一次）

4.4 需新建的基础设施清单

层次	新建组件	位置
infrastructure/	JNPF.Extras.IoT.Mqtt — MQTTnet Client + Broker 配置	新增项目
infrastructure/	JNPF.Extras.IoT.TDengine — TDengine 连接 + Repository	新增项目
modularity/iot/	JNPF.IoT — 设备管理/遥测/告警/指令 Service	新增三件套
modularity/iot/	JNPF.IoT.Entitys — 实体	新增
modularity/iot/	JNPF.IoT.Interfaces — 接口	新增
application/	IoTHub : Hub — SignalR 实时推送	Startup 注册
前端	ECharts 实时图表组件 + 告警面板	新增

本节核心表清单

表名	说明	存储
IOT_DEVICE	设备注册表	SQL Server
IOT_DEVICE_GROUP	设备分组	SQL Server
IOT_POINT_DEFINE	测点定义 (名称/单位/类型/阈值)	SQL Server
IOT_ALARM_RULE	告警规则	SQL Server
IOT_ALARM_RECORD	告警记录	SQL Server
IOT_COMMAND_LOG	指令日志	SQL Server
iot_telemetry_{deviceId}	遥测时序数据 (超级表)	TDengine

本节关键代码路径索引

路径 (规划)	说明
infrastructure/JNPF.Extras.IoT.Mqtt/	MQTT 客户端基础设施
infrastructure/JNPF.Extras.IoT.TDengine/	时序数据库访问层
modularity/iot/JNPF.IoT/	IoT 业务 Service
modularity/iot/JNPF.IoT/Hub/IoTHub.cs	SignalR Hub

第五章：需求 7 架构决策 — 升级扩展 vs 另建框架

5.1 两条路径对比

维度	方案 A：现有架构升级扩展	方案 B：另建 IoT 版低代码框架
工期	约 12-16 周 (三期范围)	6-12 个月 (从零建框架)
人力	现有团队 + 1-2 人 IoT 经验	需独立团队 5+ 人
代码复用	DI/Auth/ORM/EventBus/打印/编号/流程/大屏全部复用	重复建设 80% 通用能力
维护成本	一套代码库	两套代码库长期分叉
MES 业务	增量模块，零改框架	需在新框架重建
OA + IoT 混合项目	同一系统内 OA 审批+设备监控	两个系统间集成
技术风险	IoT 新增部分风险可控	新框架全栈风险
产品定位	"低代码 + IoT" 一体化平台	两个产品线

5.2 决策矩阵



Syntax error in text
mermaid version 10.6.1

5.3 结论：选择方案 A — 在现有架构上升级扩展

理由：

- 1. MES 业务（需求 1-5）完全不需要动框架——新增模块 + 配置即可
- 2. IoT 基础设施（MQTT/TSDB/设备管理）是旁路新增——不需要改 DI/ORM/Auth/Filter 等核心
- 3. SignalR 框架骨架已存在——只需创建 Hub 实现并在 Startup 注册
- 4. EventBus 已支持 RabbitMQ——升级为 Topic Exchange 是配置级改动
- 5. 大屏模块已有 7 个 Service——补全实时推送是增量工作
- 6. 团队已熟悉现有架构——另建框架的学习成本和风险远高于扩展

5.4 不需要做的事（避免过度设计）

不需要做

重写 DynamicApiController
重写 SqlSugar ORM
重写低代码引擎
重写代码生成器
重写前端框架
另建移动端框架

原因

IoT Service 可直接遵循现有约定
关系数据仍用 SqlSugar，时序数据用独立 TSDB 客户端
MES 表单用 VisualDev，设备面板用 VisualData 大屏
可扩展新模板，不需要改 Velocity 引擎
Vue2/3 均可嵌入 ECharts 实时组件
uni-app + SignalR 客户端即可

本节核心表清单

无（决策分析章节）。

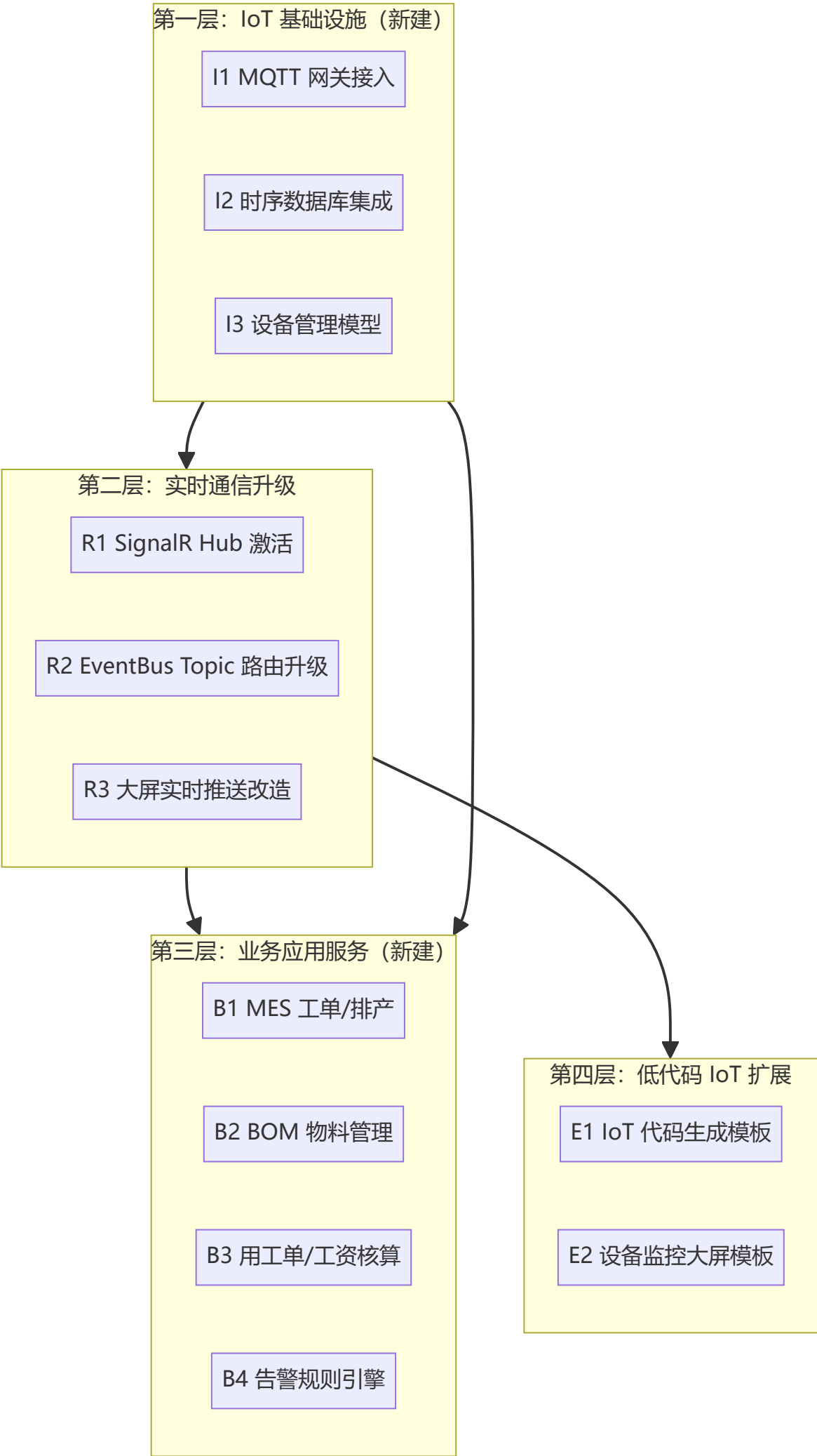
本节关键代码路径索引

无（决策分析章节）。

第六章：三期核心架构增强清单

6.1 增强项分类

基于 7 个需求的差距分析，三期架构增强分为 4 层 12 项：



6.2 逐项详细评估

# 增强项	性质	影响范围	是否改现有代码	优先级
I1 MQTT 网关接入	新建	infrastructure/ 新增项目	否	P0
I2 时序数据库集成	新建	infrastructure/ 新增项目	否	P0
I3 设备管理模型	新建	modularity/iot/ 三件套	否	P0
R1 SignalR Hub 激活	新建+配置	Startup.cs 加 AddSignalR + MapHubs	Startup 2 行	P0
R2 EventBus Topic 路由升级	升级	Startup.cs RabbitMQ 配置	Startup 配置改	P1
R3 大屏实时推送改造	升级	VisualData + 前端	ScreenService 扩展	P1
B1 MES 工单/排产	新建	modularity/mes/	否	P0
B2 BOM 物料管理	新建	modularity/mes/	否	P0
B3 用工单/工资核算	新建	modularity/mes/	否	P1
B4 告警规则引擎	新建	modularity/iot/	否	P1
E1 IoT 代码生成模板	新建	wwwroot/Template/IoT/	否	P2
E2 设备监控大屏模板	新建	VisualData 模板库	否	P2

关键指标：12 项增强中，仅 R1（2 行 Startup 配置）和 R2（RabbitMQ 配置）需要改现有代码，其余全部是新增项目/模块。

6.3 对现有代码的影响清单（极小）

文件	改动	行数
Startup.ConfigureServices	增加 services.AddSignalR()	+1
Startup.Configure	增加 endpoints.MapHubs()	+1
Startup.ConfigureServices	RabbitMQ 改 Topic Exchange + 多队列	~10 行配置
JNPF.API.Entry.csproj	增加 ProjectReference 引用 iot/mes 模块	+4
EventBus.json	增加 Topic 路由配置	配置文件

合计改动现有代码：约 15 行 + 1 个配置文件。

本节核心表清单

见各增强项中的表设计。

本节关键代码路径索引

路径	增强项
application/JNPF.API.Entry/Startup.cs	R1/R2
application/JNPF.API.Entry/JNPF.API.Entry.csproj	模块引用
application/JNPF.API.Entry/Configurations/EventBus.json	R2

第七章：低代码引擎 IoT 扩展评估

7.1 现有低代码引擎能力边界

能力	VisualDev 引擎	IoT 需求	适配度
页面范式	表单字段 → 数据库列	画布布局 + 实时指标卡片	低
数据模型	强绑定 SQL 增删改查	时序流、MQTT 推送	低
控件体系	input/select/table/iframe 等表单控件	ECharts/Gauge/实时曲线	低
写操作	Create/Update/Delete 落库	指令下发 + ACK + 状态机	低
WebType=4 数据视图	只读列表，绑定 DataInterface	设备状态列表	中
自定义按钮	customBtnsList + ExcuteInterfaceData	单次指令触发	中

7.2 扩展策略（三条路径）

路径	适用场景	改造量	推荐度
A: VisualData 大屏 + IoT 后端	设备实时监控面板	中（前端组件 + SignalR）	推荐
B: VisualDev + DataInterface	设备状态列表 + 指令按钮	低（配置为主）	辅助
C: 新增 IoT 代码生成模板	标准化设备监控页面生成	高（Velocity 模板开发）	后期

7.3 代码生成器扩展方向

扩展项	说明	优先级
Template/IoT/DeviceList.vue.vm	设备列表页（状态指示灯 + 在线/离线）	P2
Template/IoT/DeviceMonitor.vue.vm	设备监控面板（ECharts 实时图 + 指标卡）	P2
Template/IoT/AlarmList.vue.vm	告警列表（实时刷新 + 确认/处理）	P2
Template/IoT/CommandPanel.vue.vm	指令控制面板（按钮 + 回执状态）	P2
Template/IoT/Service.cs.vm	后端 Service 模板（含 TSDB 查询）	P2

7.4 移动端（APP）扩展

场景	方案
工单/用工单 APP	现有 uni-app + appForm.vue.vm 代码生成
设备告警推送	SignalR + uni-app 客户端 (@microsoft/signalr)
手环数据查看	新建 APP 页面，SignalR 实时 + TDengine 历史查询
指令下发	APP 按钮 → API → MQTT publish

7.5 不需要颠覆重写的论证

组件	是否重写	理由
后端代码生成	否	新增 Template/IoT/ 模板目录，Velocity 引擎不变
前端代码生成	否	同上，新增 Vue 模板
移动端代码生成	否	同上，新增 appIoT*.vue.vm 模板
VisualDev 引擎	否	MES 表单继续用 VisualDev；设备面板走 VisualData
VisualData 大屏	小幅增强	补全 SignalR 推送 + 实时图表组件
DynamicApiController	否	IoT Service 遵循同一约定
SqlSugar ORM	否	关系数据不变；TSDB 用独立客户端

本节核心表清单

无（评估分析章节）。

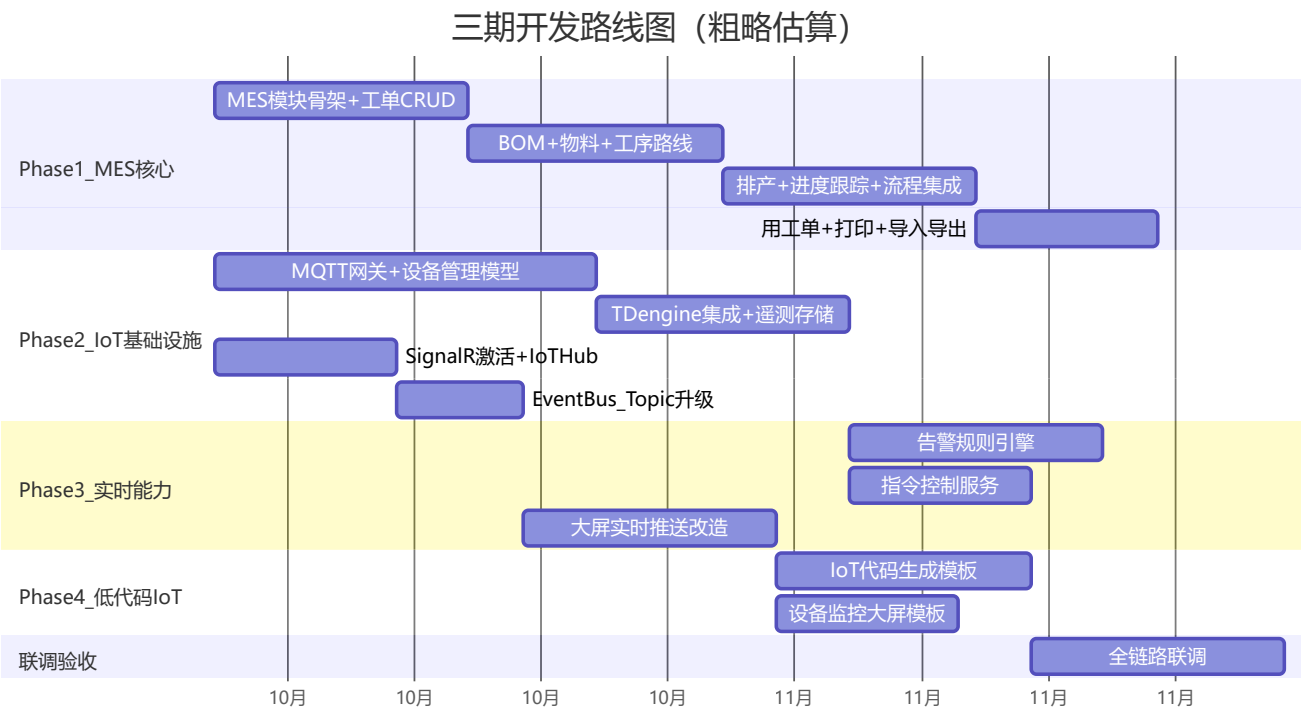
本节关键代码路径索引

路径	说明
modularity/visualdev/JNPF.VisualDev/RunService.cs	低代码运行时控件
modularity/engine/JNPF.VisualDev.Engine/Security/FrontEnd/CodeGenFormControlDesignHelper.cs	→Vue映射模板路径
modularity/engine/JNPF.VisualDev.Engine/Security/CodeGenTargetPathHelper.cs	大屏Service
modularity/visualdata/JNPF.VisualData/ScreenService.cs	

第八章：实施路线图与工期估算

声明：本章为架构选型评审阶段的粗略估算，实际开发计划待所有技术选型完成后统一制定。

8.1 三期分阶段规划



8.2 工期估算汇总

阶段	内容	估算工期	并行关系
Phase 1	MES 核心（工单/BOM/排产/用工单/打印）	7-8 周	与 Phase 2 并行
Phase 2	IoT 基础设施（MQTT/TDengine/SignalR/EventBus）	5-6 周	与 Phase 1 并行
Phase 3	实时能力（告警/指令/大屏）	4-5 周	依赖 Phase 2
Phase 4	低代码 IoT 扩展（模板/大屏）	3-4 周	依赖 Phase 3

阶段	内容	估算工期	并行关系
联调验收	全链路端到端测试	2 周	依赖全部
总计	—	约 12-16 周 (3-4 个月)	最长路径 P2→P3→P4→联调

8.3 人力建议

角色	Phase 1	Phase 2	Phase 3	Phase 4
后端 A (MES 方向)	工单/BOM/排产	—	—	—
后端 B (IoT 方向)	—	MQTT/TDengine/设备	告警/指令	IoT 模板
后端 C (平台增强)	—	SignalR/EventBus	大屏推送	—
前端	MES 页面	—	实时图表/告警面板	大屏模板
运维/DBA	—	EMQX/TDengine 部署	—	—
QA	—	—	—	联调验收

8.4 技术选型清单（待审批）

领域	推荐选型	备选	决策依据
MQTT Broker	EMQX 开源版	MQTTnet 内嵌	百万连接/规则引擎/Dashboard
MQTT 客户端	MQTTnet	—	.NET 原生/NuGet 标准
时序数据库	TDengine	InfluxDB	SQL 兼容/国产化/超级表
实时推送	SignalR (框架骨架已有)	—	.NET 原生/Hub 自动注册
消息队列	RabbitMQ Topic Exchange (已有)	Kafka	已集成/升级成本低
排产算法	规则排产 (L2)	APS 独立服务	三期先 MVP
前端图表	ECharts (已有)	—	大屏已用

8.5 风险与缓解

风险	概率	影响	缓解措施
TDengine 与 .NET 生态兼容性	中	高	先做 PoC 验证 ADO.NET 连接
EMQX 运维复杂度	低	中	先单节点，后期集群
秒级数据量超预期	中	高	TDengine 降采样 + 冷数据归档
前端实时图表性能	中	中	SignalR 聚合推送 (降频到 1-5 秒)
排产算法复杂度	高	中	三期仅 L1+L2, APS 后期专项

附录 A：项目场景与架构能力映射

场景	MES 模块	IoT 基础设施	实时通信	低代码引擎
机加工 MES	工单/BOM/排产/用工单/打印	质检设备数据采集	进度看板	VisualDev 表单
钢铁 MES	工单/工序/质检	炼钢炉/轧钢/拉索传感器	实时大屏+告警	VisualData 大屏
智慧工地	考勤工单	塔吊/升降机/人脸终端	实时监控+指令控制	VisualData 大屏
智能手环	—	手环健康数据	APP 实时+告警	APP 页面

附录 B：MQTT Topic 规范草案

Topic	方向	QoS	说明
device/{deviceId}/telemetry	上行	0-1	遥测数据

Topic	方向	QoS	说明
device/{deviceId}/status	上行	1	在线/离线/故障
device/{deviceId}/alarm	上行	1	设备端告警
device/{deviceId}/command	下行	1-2	控制指令
device/{deviceId}/command/response	上行	1	指令回执
gateway/{gwId}/sub	上行	1	子设备注册

附录 C：TDengine 超级表设计草案

```
-- 超级表：所有遥测数据的模板
CREATE STABLE iot_telemetry (
    ts TIMESTAMP,
    value_double DOUBLE,
    value_string NCHAR(200),
    quality INT
) TAGS (
    device_id NCHAR(50),
    point_code NCHAR(100),
    point_name NCHAR(200),
    unit NCHAR(20),
    tenant_id NCHAR(50)
);

-- 子表：按设备+测点自动创建
-- CREATE TABLE iot_telemetry_{deviceId}_{pointCode} USING iot_telemetry TAGS(...)
```

深度自检清单

- ✓ 7 个功能需求逐项评估，结论明确
- ✓ 现有架构能力基线：源码事实，标注文件路径
- ✓ 差距分析：12 项增强清单，标注优先级
- ✓ 架构决策：方案 A/B 对比，结论明确
- ✓ 技术选型：MQTT/TSDB/排产等均有推荐+备选+依据
- ✓ 对现有代码的影响：约 15 行 + 1 个配置文件（极小）
- ✓ 不需要重写的论证：7 个组件逐一说明
- ✓ 实施路线图：甘特图 + 分阶段工期
- ✓ Mermaid 图表：6 张（架构图/数据流/状态机/ER/甘特/决策矩阵）
- ✓ 核心表清单：MES 6 张 + IoT 7 张
- ✓ 代码路径索引：各章节末尾均有
- ✓ 【待源码验证】标注：TDengine .NET 连接性需 PoC

文档遵循 [docs/ARCHITECTURE_DOC_RULES.md](#) 编写。